Form 146 Copilot: drafting treaty withholding certificates for Indian payments abroad

Brief for the Tresata take-home · India, Income-tax Act 2025 · treaties with the US, UK, Singapore, Germany and the Netherlands · Prototype form146-copilot.vercel.app · Code github.com/mohit-gautam7/form146-copilot

1. The area, and what surprised me

When an Indian company pays a non-resident for software, consulting or a licence, it must withhold Indian tax before the money leaves. If the payment is taxable and above ₹5 lakh, a chartered accountant (CA) certifies what the payment is, which treaty article applies and the rate. Since 1 April 2026 that certificate is **Form 146** (formerly 15CB), under **section 393(2)** of the Income-tax Act 2025 (formerly section 195).

- **The same service is taxed differently by treaty.** Consulting that leaves no lasting know-how with the payer is "fees for technical services" at 10% under the India–Germany treaty, but business profits (no Indian tax without a PE) under the US, UK and Singapore treaties, because they add a "make available" test. The answer turns on contract wording, not the invoice label.
- **One payment can straddle two Acts.** TDS follows the earlier of credit and payment; the remittance forms follow the payment date (Income Tax Department FAQs). An amount credited on 25 March 2026 and wired on 9 April is taxed under the 1961 Act but reported on Form 146.
- **Treaties change quietly.** The first plan assumed the Netherlands treaty had no make-available test; the official text shows a 1999 notification inserted one. And an MFN clause does not apply by itself without a notification (Supreme Court, *Nestlé*, 2023).
- **From the practitioner interview:** the slow part is not arithmetic. Active time goes into reading the contract and statement of work; elapsed time goes into chasing tax residency certificates (TRC) and Form 41.

2. The problem

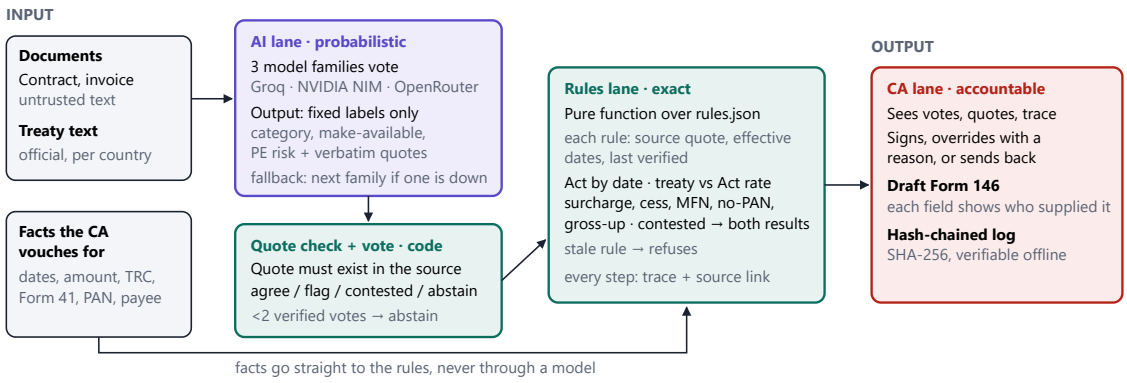
Who	A CA who certifies foreign remittances, and the payer's finance team waiting for the certificate.
The task	Read the contract and invoice; decide what the payment is (royalty, technical services, business profits, reimbursement); check the treaty article and its conditions; compute the rate; sign Form 146.
How often	The CA I interviewed signs **25–40 certificates a month**.
What it costs today	**30–45 minutes** for a repeat vendor, **2–4 hours** of active work for a new contract, **2–5 days** elapsed when documents are missing. A wrong certificate leaves the payer liable for the unpaid tax with interest and puts the CA's signature at risk.
Why now	The new Act and forms started on 1 April 2026. Templates are being moved from 15CB to 146, and payments credited before and paid after the change follow two Acts at once.

Problem statement. Working out whether a cross-border payment is a royalty, technical services or business profits under the right treaty is slow judgement work, repeated from scratch for every certificate. Since April 2026 it must be done under renumbered law while old templates are still in circulation, and a confident mistake costs the payer tax and interest.

Out of scope: filing on the tax portal and generating the UDIN; profit attribution to a permanent establishment (PE); beneficial-ownership, principal-purpose and multilateral-instrument analysis; the yearly aggregate test for the ₹5 lakh threshold (applied per payment); countries beyond the five treaties; PDF upload (text only); tax advice: the CA decides.

3. Solution design

The design splits the work by what each part can be trusted with: **models read**, **code computes**, **a CA decides**. The three lanes never mix: a model never produces a rate, and code never decides what a contract means.



Data in, answer out

In: contract and invoice text; the treaty articles for the payee's country (fetched from the Income Tax Department site); facts the CA vouches for (credit and payment dates, amount, TRC, Form 41, PAN, payee type, who bears the tax). **Out:** a rate with its basis and article, the tax, the governing Act and forms, a step-by-step trace with sources, the model votes with their quotes, and a draft Form 146 for the CA.

Handling uncertainty and mistakes

- A vote counts only if it quotes both the contract and the treaty, and the quotes are found word for word in the sources.
- Fewer than two counted votes: **abstain**. A split between technical services and business profits is never settled by majority: it is **contested**, and the engine computes every reading for the CA. A make-available dispute that cannot change the tax is not escalated.
- A model that is down is replaced by one from another family; an outage never counts as a vote.
- A stale or out-of-window rule makes the engine refuse. Rules not yet verified against a primary source are flagged on every result that uses them.

Why AI is needed, and only here

The hard step is reading free text: a "subscription" that grants rights to reproduce and sublicense software is a royalty; "training our engineers to run the pipeline" makes know-how available, while a research report does not. Keyword rules fail both ways. Everything an auditor checks (rates, dates, thresholds, arithmetic) is code, and the CA signs.

Where the knowledge lives

- **Treaty text:** official texts stored in the repo with source links.
- **Rates and rules:** `rules.json`, each with a source quote, effective dates and a last-verified date. Older than a year or outside its window: the engine refuses.
- **Models:** reading skill only, never the law.
- **The CA:** judgement and accountability.

Traceability

Each rate step links to its source and quote; each model claim has a verified quote; raw model replies are stored; the CA's decisions sit in a hash-chained log that an auditor can re-verify offline.

Alternatives I considered and rejected

Alternative	Why not
One LLM call returns the rate	Rates and arithmetic would not be exact or auditable, and the model cannot know a rule changed last month.
Retrieval with embeddings	The corpus is tiny (four articles per treaty). Looking them up by country is exact and auditable; similarity search adds a way to fetch the wrong article.
One model checking itself	Its errors correlate with itself. Three model families at least cancel random errors.
Several cooperating agents	The workflow is fixed. Agents add failure modes without adding accuracy.
Rules or keywords only	Cannot read contracts; fails the "SaaS that grants copyright" test case.
Majority vote wins	Rejected after evidence: in the first live run a 2–1 majority produced a wrong nil rate shown as final.
Escalate a split only if the readings give different tax	Tried, and rescored on the recorded answers: it fixed one over-cautious case but turned a genuinely contested case into a confident nil, because all three models misread it the same way. Kept the blunter rule.
A database server	No shared data in a demo. A local hash-chained log is verifiable; Postgres when several reviewers share a queue.

4. Proof: 17 test cases, and where it fails

All contracts and invoices are synthetic; the treaty texts are real. 14 cases each target one trap (SaaS, make-available, Netherlands equipment hire, MFN claim, no TRC, gross-up, old Act, PE risk, below threshold, no PAN). 3 more were written afterwards to break it: a prompt injection in an invoice, a "SaaS" contract that grants copyright, and a credit/payment straddle across 1 April 2026. Expected answers were fixed before the models saw the cases. The headline metric is **wrong and confident**: a wrong rate shown as final instead of being sent to the CA.

Run (live free-tier models, 2 Nov 2026)	Fully correct	Wrong and confident	Notes
Rules lane, correct labels given	17 / 17	0	A regression check, not evidence: one author wrote rules and answers.
1: first 14 cases	11 / 14	**1**	2–1 vote gave a wrong nil rate. Dispute rule added.
2: same 14, after the rule	10 / 14	0	3 misses were quota outages (OpenRouter's 50 a day).
3: the 3 new cases, first contact	1 / 3	0	Found a gap: a dispute that cannot change the rate was escalated.

All 17, after that fix (what the site replays)	**12 / 17**	**0**	Two fixes so far were made on cases already seen.

Where it fails. All five misses went to the CA rather than giving a wrong rate. Four trace back to free-tier outages that left one verified vote. One (a UK research report) is the dispute rule being over-cautious: the models disagreed only on the label, and both labels give nil. Narrowing the rule to fix it was tested on the recorded answers and rejected (see the table above). In the injection case neither model obeyed the injected instruction or used the fake quote, but one model's treaty quote was not found, so it abstained. Seventeen cases cannot show the rules generalise.

5. Risks and how the design handles them

Risk	Handling
Wrong answers	Three-family vote, verbatim quote check, abstain and contested paths, both outcomes computed. Measured by "wrong and confident", not accuracy. The CA signs.
Outdated rules	Effective dates and last-verified dates on every rule; stale rules make the engine refuse. Two rules are still unverified (no surcharge on treaty rates; the no-PAN carve-out under the 2025 Act) and are flagged on every result. Surcharge slabs and the 2025 rate are checked against the Finance Bill 2026; the enacted Act should be compared.
Confidential data	Real contracts would go to third-party APIs. The prototype uses only synthetic data and the public site does not call models. For a firm: redact names and amounts (the classification does not need them), use zero-retention terms, or self-host an open model.
Liability	The law requires a CA's certificate, so the CA is accountable. The tool drafts and never shows "approved"; every Form 146 field shows whether the model, the code or the CA supplied it; overrides need a reason.
Prompt injection	Documents are fenced as evidence; a model cannot set a rate; quotes must exist; with no TRC even an obeyed "business profits" goes to the CA. Tested once; one naive attack proves little.
Weakest point	Correlated error: three models can share one misconception, and a wrong rule in `rules.json` is applied consistently. Voting does not catch either. Seen once already: on the contested Singapore case all three models read nil, and only the blunt dispute rule sent it to the CA. What would break it: a treaty amendment that is not yet in the rules file, or a contract style none of the test cases cover.

6. Before spending more time on this

- Interview three to five more CAs and a bank's remittance desk; time real, anonymised certificates with and without the tool.
- Have practitioners write the test cases and expected answers, so the author of the rules is not also the examiner.
- Model the yearly aggregate (a payment ledger) and the AO-certificate route, where Form 146 is not needed.
- Add a challenge pass for every nil result: ask a model what clause could make the payment taxable.
- Decide how contracts stay confidential (redaction or self-hosting) before any real contract touches it.
- **Another jurisdiction:** for a US payer the inputs become Form W-8BEN-E, withholding under chapter 3 and Form 1042-S, and the AI question becomes "US-source income or not?". The rules file changes; the three lanes do not.

Appendix: what I added beyond the AI

I used Claude (Claude Code) for research, architecture options, most of the code, the evaluation harness and drafting this brief. Groq, NVIDIA NIM and OpenRouter models are the ones under test inside the prototype. The interview record (Annex A) and the full build chat (Annex B) follow this page.

Where AI helped

Finding and reading primary sources (the 2025 Act mapping, the treaty texts, the Income Tax Department FAQs, the Finance Bill schedule); proposing the three-lane split, the quote check and the evaluation metric; writing the Next.js prototype and tests; drafting and editing this document. Three of my prompts, verbatim:

> "*research for this and give me some good ideas for this … think, research, and give me a best solution*"

> "*add more option also like nvidia nim,groq,openrouter all 3 and chached replay also so give me best running*"

> "*make sure to add option of fallback if any model is not available then you can use other*"

What I rejected or corrected, and how I knew

- **The Netherlands assumption.** The plan treated the India–Netherlands treaty as having no make-available test. The official text shows Notification S.O. 693(E) of 30 August 1999 inserted it, so that test case was changed to equipment hire, which the Dutch royalty definition excludes.
- **Majority voting.** The first live evaluation showed a 2–1 majority producing a wrong nil rate presented as final. Majority-wins was dropped for splits that change the tax.
- **"One unfound quote voids the vote".** Too strict: a non-breaking hyphen or an ellipsis threw away correct answers. Replaced by "at least one verified quote from the document and one from the treaty", with unfound quotes shown in red.
- **A wrong test, not wrong code.** A test expected 21.216% on ₹20 lakh; surcharge only starts above ₹1 crore, so the engine's 20.8% was right.
- **An outage that looked like a vote.** I noticed the workbench showing a raw HTTP 429 error as if it were a model's answer. It now says "No model answered" with the reason.
- **Claims in my own research notes** (credit vs payment date, Form 41, the surcharge slabs) went into the code only after they were checked against the Income Tax Department FAQs and the Finance Bill 2026.

What came from me

- **The practitioner interview.** The CA said the statement of work, not the invoice label, decides the classification; that time goes into reading contracts and chasing documents; that they would not sign without clause-level evidence and a calculation they can reproduce; and that treaty rates are applied without surcharge and cess. These shaped the quote requirement, the source trace on every step, the rule that nothing is ever marked "approved", and the treaty-rate rule.
- **Three model providers, a fallback chain and recorded replay,** so free-tier limits never break the public demo. Run 2 showed why: OpenRouter's daily cap ran out mid-run.
- **Three test cases written to break it:** an injection in an invoice, a "SaaS" contract that grants copyright, and a credit in March with payment in April 2026. The last one exposed a gap nobody had flagged: the tool escalated a dispute that could not change the rate under the Germany treaty.

One thing I would defend live without tools

Models read, code computes, a CA decides. The rate is a lookup and arithmetic over published law, so it must be exact and traceable, and that belongs in code with a source for every number. What the payment *is* depends on reading contract language, which is where models help and where they can be wrong. So their output is limited to fixed labels backed by quotes that must exist in the documents, and any disagreement that would change the tax goes to the CA with both answers computed. Three models agreeing is not proof: they can share one misconception, which is why the CA signs and why the measure that matters is how often the tool is wrong and confident, not how often it is right.